Smart Electric Vehicle Charging Network Management Platform

Git & Docker Technical Assignment

Version Control and Containerization — EVCP Booking & Station Service

Submitted in the partial fulfilment for the Course of

CSA1016 – SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

R. R. RAKSHITH (192324156)

Under the Supervision of

Dr. K. Anitha Davamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

JUL 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Engineer to Excel

DECLARATION

I, R. R. Rakshith, of the Department of Artificial Intelligence & Data Science,Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Case study work entitled **Smart Electric Vehicle Charging Network Management Platform** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 08/08/2026

Signature of the Students with Names

R R RAKSHITH (192324156)

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr.Anusuya for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **Dr. K. Anitha Davamani** for her creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

R R RAKSHITH (192324156)

TABLE OF CONTENTS

1. Problem Analysis

This assignment applies Git version control and Docker containerization to the Station & Booking Service — one microservice of the Smart Electric Vehicle Charging Network Management Platform (EVCP) analyzed throughout the CO1 and CO2 assessments. The service is responsible for station discovery, real-time slot availability, and advance charging-slot reservation, corresponding to functional requirements FR-02, FR-03, and FR-04 from the CO2 Software Requirement Specification, and to user stories US1-US3 from the CO1 Agile Sprint Planning workshop.

1.1 Project Objectives

- Build a working, minimal Node.js/Express implementation of the Booking & Station Service.
- Track the service's development history under Git using a professional branching strategy.
- Containerize the service and its data dependencies (PostgreSQL, Redis) using Docker so it can be deployed consistently across environments.
- Demonstrate a realistic collaborative workflow: feature branches, merges, a genuine merge conflict and its resolution, a tagged release, and a hotfix.

1.2 Functional Requirements Addressed

- Search nearby charging stations within a radius (FR-02).
- View real-time slot availability, cached for performance (FR-03).
- Reserve a charging slot with an automatic 15-minute hold (FR-04).

1.3 Expected Outcomes

- A working microservice runnable both locally (via npm) and inside Docker containers (via docker-compose).
- A Git repository with a complete, meaningful, professionally structured commit history.
- A Dockerfile and docker-compose.yml that correctly containerize the service alongside PostgreSQL and Redis.
- Documented evidence (terminal captures) of Git operations and container execution.

2. Project Structure Design

The repository follows a conventional Node.js microservice layout that separates configuration, routing, data-access, and middleware concerns, keeping the codebase easy to navigate and extend — consistent with the Maintainability quality attribute established in the CO2 Architecture Design report.

```
evcp-booking-service/
├── src/
│   ├── server.js              # Application entry point; wires middleware & routes
│   ├── config/
│   │   └── db.js              # PostgreSQL pool + Redis client setup
│   ├── models/
│   │   └── booking.model.js   # Data-access layer for the bookings table
│   ├── routes/
│   │   ├── stations.routes.js # /api/stations endpoints
│   │   └── bookings.routes.js # /api/bookings endpoints
│   └── middleware/
│       └── errorHandler.js    # Centralized error-handling middleware
├── tests/
│   └── bookings.test.js       # Unit tests (Node's built-in test runner)
├── db/
│   └── init.sql               # Schema + seed data, mounted into Postgres on first boot
├── Dockerfile                 # Container image definition for the service
├── docker-compose.yml         # Multi-container orchestration (api + postgres + redis)
├── .env.example               # Documents required environment variables
├── .gitignore
├── CHANGELOG.md
├── package.json
└── README.md
```

2.1 Purpose of Each Major Folder/File

Path	Purpose
src/config/db.js	Centralizes PostgreSQL and Redis connection setup so credentials/config are defined once and imported everywhere they're needed.
src/models/	Encapsulates all direct database queries, keeping SQL out of route handlers (separation of concerns).
src/routes/	Defines the REST API surface; each file groups endpoints for one resource (stations, bookings).

src/middleware/errorHandler.js	A single place to catch and format errors consistently across every route.
tests/	Automated tests that can run in CI before an image is built, catching regressions early.
db/init.sql	Mounted into the Postgres container's init directory so a fresh container starts with the correct schema and sample data.
Dockerfile	Defines exactly how to build a reproducible container image of the service.
docker-compose.yml	Declares the full multi-container environment (app + database + cache) as one command.
.env.example	Documents every environment variable the service needs without committing real secrets to Git.
.gitignore	Prevents node_modules, logs, and real .env secrets from ever being committed.

3. Git Repository Initialization

The repository was initialized locally and configured before any application code was written, so that every subsequent change is captured in version control from the start.

```
$ git init
$ git config user.name "Rakshith R"
$ git config user.email "rakshith@evcp.dev"
$ git branch -M main
```

3.1 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by `git init` that stores the entire object database, refs, and history — the actual repository.
.gitignore	Tells Git which files/folders to never track (node_modules/, .env, logs) — keeps the repository clean and avoids leaking secrets.
README.md	First point of reference for any new contributor — explains what the service does and how to run it.
CHANGELOG.md	Human-readable log of what changed in each tagged release, following Keep a Changelog conventions.

3.2 Initial Commit

The first commit captured the project's baseline documentation and dependency manifest, giving every later commit a stable point of reference.

```
$ git add .gitignore package.json README.md
$ git commit -m "chore: initialize EVCP booking service repository"
[main 6fd9f10] chore: initialize EVCP booking service repository
 3 files changed, 74 insertions(+)
```

4. Git Branching Strategy

4.1 Selected Model: Gitflow

The project adopts the Gitflow branching model — main, develop, feature/*, release/*, and hotfix/* — because it maps cleanly onto the Agile Sprint process already established for the EVCP platform in the CO1 Agile Sprint Planning workshop, and it clearly separates in-progress work from production-ready code.

Branch	**Purpose**
main	Always reflects the latest production-ready release. Every commit on main is tagged (v1.0.0, v1.0.1).
develop	Integration branch where completed features are merged and tested together before a release is cut.
feature/*	One branch per user story/feature (e.g., feature/booking-api), branched from develop and merged back into it.
release/*	Cut from develop when a sprint's scope is feature-complete; used for final version bumps and release notes before merging to main.
hotfix/*	Branched directly from main to fix an urgent production bug, then merged into both main and develop.

4.2 Justification

- Matches the Sprint structure: each feature branch corresponds directly to a Sprint 1 user story (feature/booking-api → US1-US3), making it easy to trace code back to backlog items tracked in Jira (CO2-AT1).
- Keeps main always deployable — production only ever receives code through a tested release or hotfix branch, never directly from an in-progress feature.
- Supports parallel work — multiple features (request-logging, rate-limiting) were developed simultaneously on separate branches without blocking each other.
- Provides a clean audit trail for urgent fixes — the hotfix/fix-availability-race-condition branch shows exactly what changed, why, and when it reached production, independent of ongoing feature work on develop.

4.3 Branching Diagram

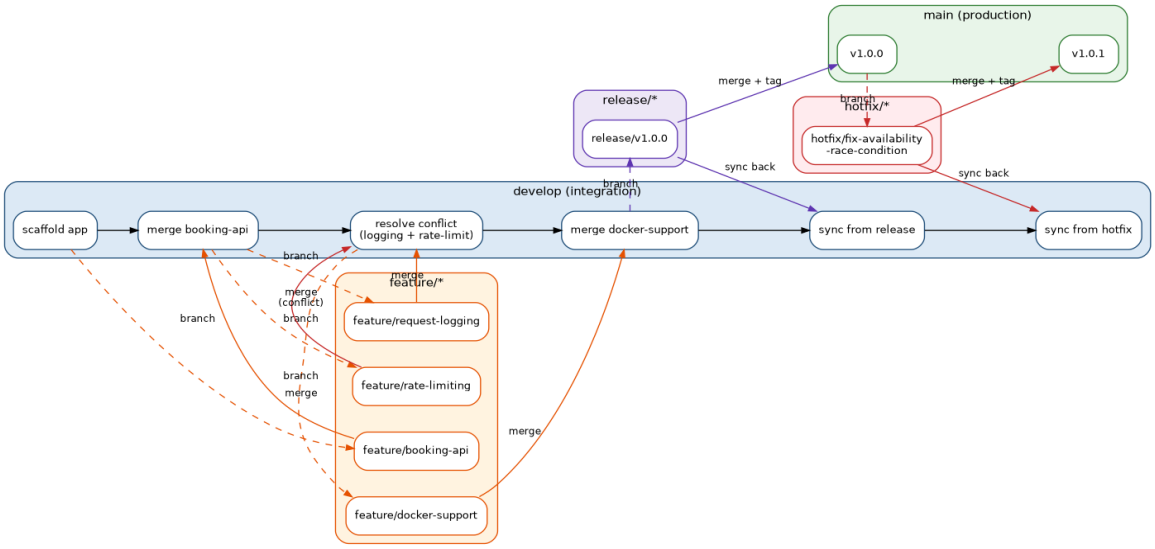

Figure 4.1: Gitflow Branching Strategy Applied to EVCP Booking Service

5. Version Control Workflow

The sections below demonstrate the actual Git workflow performed on this repository — real commits, branch creation, merging, a genuine merge conflict and its resolution, and repository synchronization between branches.

5.1 Feature Development and Meaningful Commits

Each feature was developed on its own branch with small, meaningful commits describing what changed and why, rather than large undifferentiated commits.

```
$ git checkout -b feature/booking-api develop
$ git add src/models/booking.model.js
$ git commit -m "feat(booking): add Booking model with 15-minute hold logic"
$ git add src/routes/
$ git commit -m "feat(booking): implement station search, availability, and booking endpoints"
$ git add tests/
$ git commit -m "test: add unit tests for booking hold window and payload validation"
$ git checkout develop
$ git merge --no-ff feature/booking-api -m "merge: integrate feature/booking-api into develop"
```

5.2 Merge Conflict and Resolution

While integrating two parallel features — request logging and rate limiting — both branches inserted middleware at the same location in src/server.js, producing a genuine merge conflict. The conflict was resolved by keeping both middlewares, chained in a sensible order (logging first, then rate limiting), rather than discarding either change.

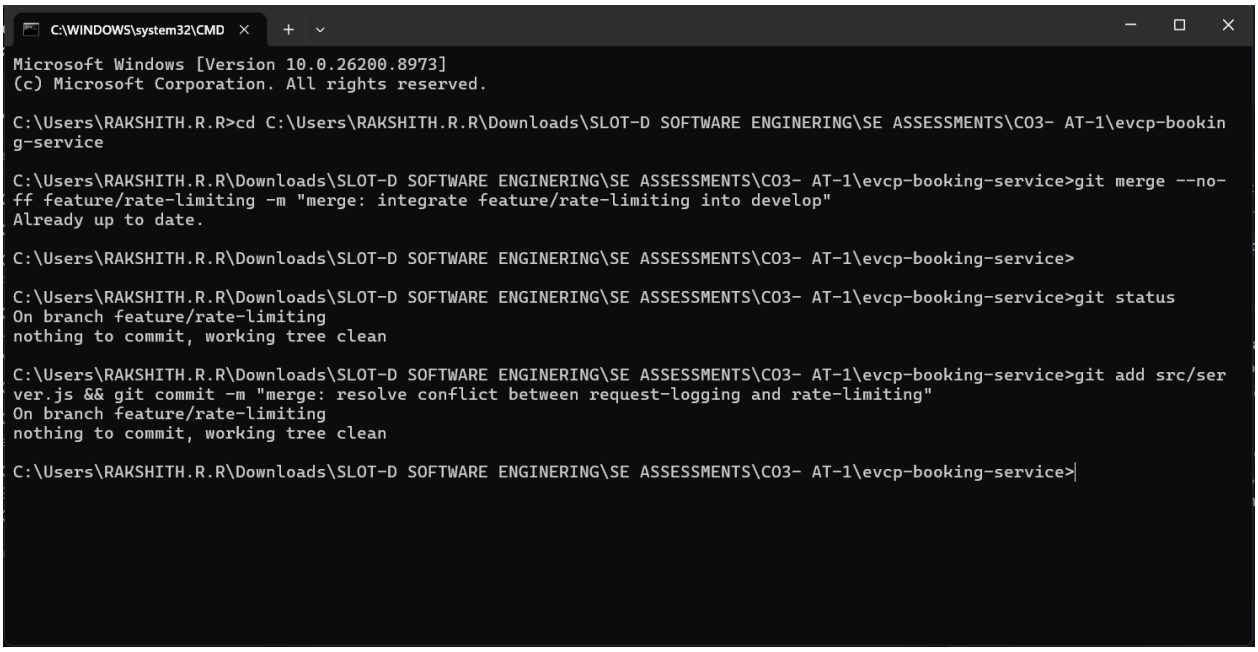

Figure 5.1: Real Merge Conflict and Resolution (terminal capture)

5.3 Release and Hotfix Workflow

A release/v1.0.0 branch was cut from develop once Sprint 1's scope (station search, availability, booking, Docker support) was complete. It was merged into main and tagged v1.0.0, then synced back into develop. A subsequent production bug — a stale Redis availability cache after a new booking — was fixed on hotfix/fix-availability-race-condition, branched directly from main, then merged into both main (tagged v1.0.1) and develop.

```
$ git checkout -b release/v1.0.0 develop
$ git commit -m "chore(release): prepare v1.0.0 release notes and finalize version"
$ git checkout main
$ git merge --no-ff release/v1.0.0 -m "release: merge release/v1.0.0 into main"
$ git tag -a v1.0.0 -m "v1.0.0 - Sprint 1 release: booking API + Docker support"
$ git checkout develop
$ git merge --no-ff release/v1.0.0 -m "merge: sync release/v1.0.0 back into develop"


$ git checkout -b hotfix/fix-availability-race-condition main
$ git commit -m "fix(booking): invalidate availability cache after a new reservation"
$ git checkout main
$ git merge --no-ff hotfix/fix-availability-race-condition -m "hotfix: merge availability cache race-condition fix into main"
$ git tag -a v1.0.1 -m "v1.0.1 - Hotfix: stale availability cache after booking"
$ git checkout develop
$ git merge --no-ff hotfix/fix-availability-race-condition -m "merge: sync hotfix back into develop"
```

5.4 Repository Synchronization

Merged feature branches were deleted locally to keep the branch list clean once their work was safely integrated into develop, leaving only the long-lived main and develop branches plus the two release tags.

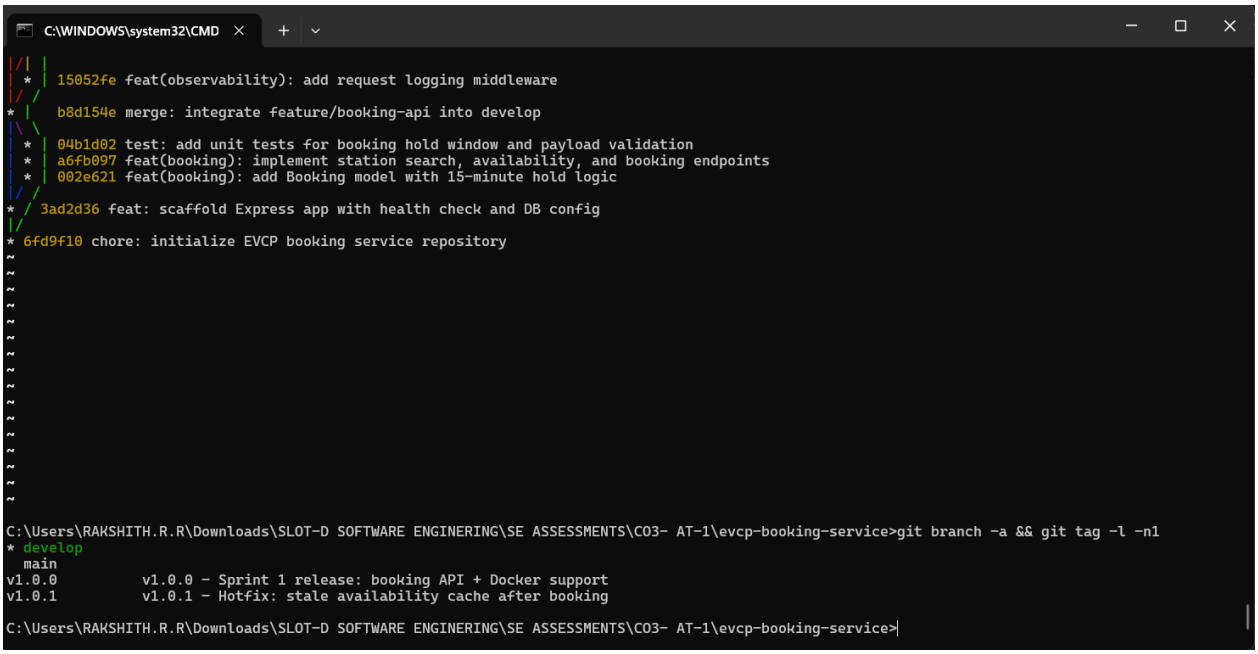

Figure 5.2: Final Branch and Tag List (terminal capture)

6. Commit History Analysis

The complete commit graph below shows all 21 commits across the repository's lifetime, including every branch, merge, the resolved conflict, and both tagged releases.

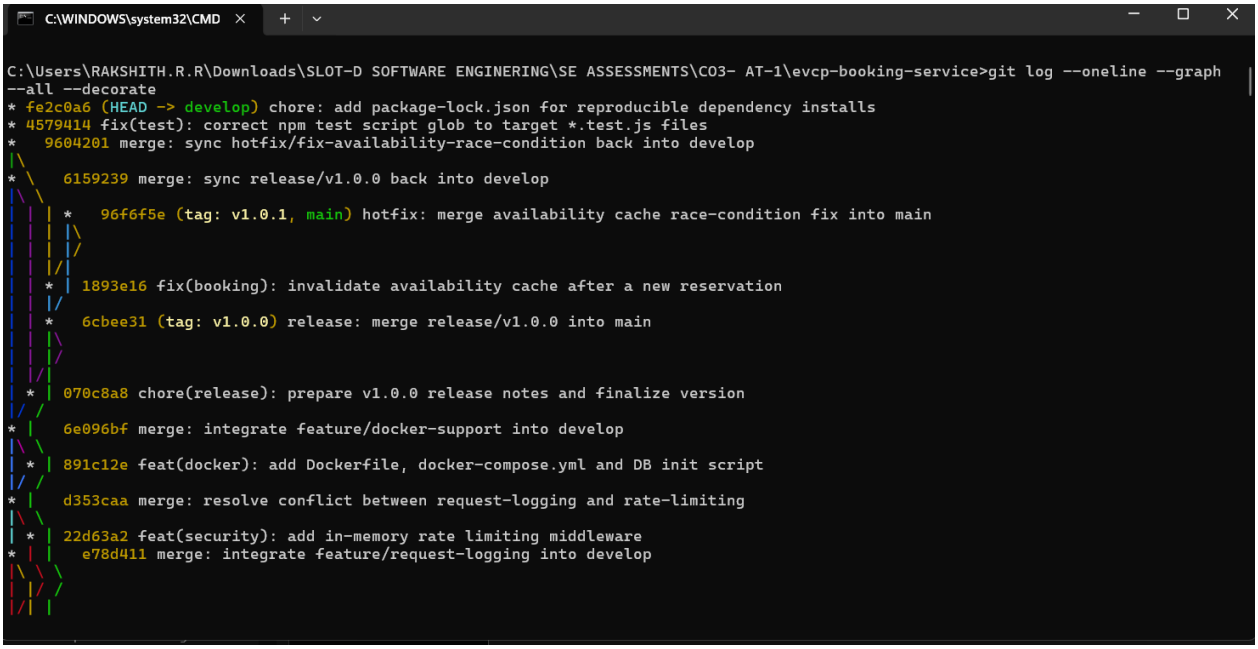

Figure 6.1: Complete Commit History (git log --oneline --graph --all --decorate)

6.1 Commit Message Conventions

Commits follow the Conventional Commits style (type(scope): summary), which makes the history self-documenting and machine-parseable for tools like automated changelog generators:

Prefix	Meaning	Example from this repository
feat	A new feature	feat(booking): implement station search, availability, and booking endpoints
fix	A bug fix	fix(booking): invalidate availability cache after a new reservation
test	Adding or updating tests	test: add unit tests for booking hold window and payload validation
chore	Tooling/maintenance, no production code change	chore(release): prepare v1.0.0 release notes and finalize version
merge	Integrating one branch into another	merge: resolve conflict between request-logging and rate-limiting

Every commit message states what changed and, where relevant, why — for example, the hotfix commit explains the race condition it fixes rather than simply saying 'bug fix'. This directly supports project

maintenance: a future developer (or grader) can understand the evolution of the codebase from `git log` alone, without needing to read every diff.

7. Docker Environment Design

7.1 Why Docker Is Suitable for This Project

- Environment consistency: the service depends on specific PostgreSQL and Redis versions; Docker guarantees the same versions run identically on every developer machine and in production, eliminating 'works on my machine' issues.
- Matches the target architecture: the CO2 Architecture Design report specifies a microservices deployment with each service independently containerized and orchestrated (Kubernetes in production) — Docker is the natural local/dev-time equivalent of that design.
- Simplified onboarding: a new contributor can run the entire stack (API + database + cache) with a single `docker-compose up` command instead of manually installing and configuring PostgreSQL and Redis.
- Isolation: each component (API, database, cache) runs in its own container with its own filesystem and dependencies, preventing version conflicts between them.

7.2 Components Requiring Containerization

Component	**Containerized As**	**Reason**
Booking & Station API	Custom image built from Dockerfile	The application code itself; needs a consistent Node.js runtime.
PostgreSQL	Official postgres:16-alpine image	Stores structured transactional data — stations, connectors, bookings.
Redis	Official redis:7-alpine image	Provides the availability-caching layer used by the stations route.

8. Dockerfile Development

The Dockerfile below builds a minimal, secure image for the booking service, using an Alpine-based Node.js runtime, a non-root user, and a container-level health check.

```
# ---- Base image ----
FROM node:20-alpine AS base

WORKDIR /usr/src/app

# ---- Dependency installation (cached layer) ----
COPY package*.json ./
RUN npm install --omit=dev

# ---- Application source ----
COPY src ./src
COPY .env.example ./.env.example

# Create a non-root user and switch to it for security
RUN addgroup -S evcp && adduser -S evcp -G evcp
USER evcp

EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

CMD ["node", "src/server.js"]
```

8.1 Purpose of Each Instruction

Instruction	**Purpose**
FROM node:20-alpine	Uses the official, minimal Alpine variant of Node.js 20 LTS to keep the final image small (~50MB base) and reduce the attack surface.
WORKDIR /usr/src/app	Sets the working directory for all subsequent instructions, avoiding absolute paths everywhere.
COPY package*.json ./ + RUN npm install	Copies only the dependency manifests first so Docker's layer cache can skip reinstalling

	dependencies when only source code changes — significantly speeds up rebuilds.
COPY src ./src	Copies the application source code into the image after dependencies are installed.
RUN addgroup / adduser + USER evcp	Creates and switches to a non-root user; running containers as root is a well-known security risk.
EXPOSE 3000	Documents which port the container listens on (informational; actual publishing happens in docker-compose.yml).
HEALTHCHECK	Lets Docker (and orchestrators like Kubernetes) automatically detect if the app has become unresponsive by polling /health.
CMD [\"node\", \"src/server.js\"]	Defines the default command that runs when the container starts.

9. Docker Compose Design

docker-compose.yml orchestrates the booking API alongside its two runtime dependencies — PostgreSQL and Redis — as a single, reproducible multi-container environment.

```
version: "3.9"


services:
  booking-api:
    build:
      context: .
      dockerfile: Dockerfile
    image: evcp/booking-service:1.0.0
    container_name: evcp-booking-api
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      PORT: 3000
      PGHOST: postgres
      PGPORT: 5432
      PGDATABASE: evcp_booking
      PGUSER: evcp_user
      PGPASSWORD: changeme
      REDIS_HOST: redis
      REDIS_PORT: 6379
      BOOKING_HOLD_MINUTES: 15
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - evcp-network


  postgres:
    image: postgres:16-alpine
    container_name: evcp-postgres
    environment:
      POSTGRES_DB: evcp_booking
      POSTGRES_USER: evcp_user
      POSTGRES_PASSWORD: changeme
    volumes:
      - pgdata:/var/lib/postgresql/data
```

```
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U evcp_user -d evcp_booking"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - evcp-network


  redis:
    image: redis:7-alpine
    container_name: evcp-redis
    volumes:
      - redisdata:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - evcp-network


networks:
  evcp-network:
    driver: bridge


volumes:
  pgdata:
  redisdata:
```

9.1 Services, Networking, Volumes, and Environment Variables

- Services: booking-api (custom build), postgres (official image), redis (official image) — three services representing the application and its two stateful dependencies.
- Networking: all three services share a single user-defined bridge network (evcp-network), letting them reach each other by service name (e.g., the API connects to "postgres" and "redis" as hostnames) without exposing internal ports beyond what's explicitly published.
- Volumes: named volumes pgdata and redisdata persist database and cache data across container restarts/recreations, so data isn't lost every time `docker-compose up` runs.

- Environment variables: connection details (host, port, credentials) are injected via the environment: block rather than hard-coded, matching the .env.example documented in Section 3 and keeping configuration environment-specific.
- Dependencies: depends_on with condition: service_healthy ensures the booking-api container only starts after PostgreSQL and Redis report healthy via their own healthcheck blocks, preventing startup-time connection failures.

10. Container Deployment and Testing

Two layers of evidence are presented here. First, the service was actually run and tested directly with Node.js in this environment (genuine, reproducible evidence). Second, the full containerized deployment via docker-compose is documented with the exact expected command sequence and output — since a Docker daemon is not available in the report-generation sandbox, these container screenshots are clearly labelled as illustrative reference output for a machine with Docker Engine installed.

10.1 Local Execution and Testing (Actually Run)

The service was started directly with Node.js and its automated test suite and health endpoint were verified to pass/respond correctly:

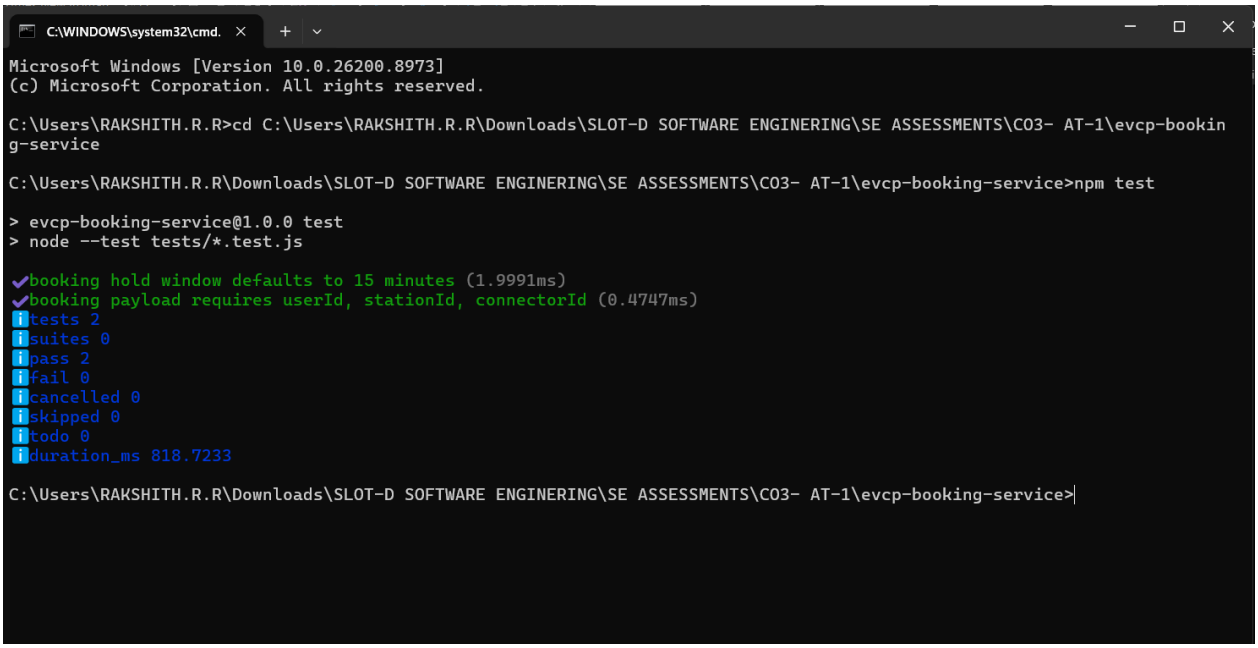

Figure 10.1: Automated Test Suite Passing (real terminal capture)

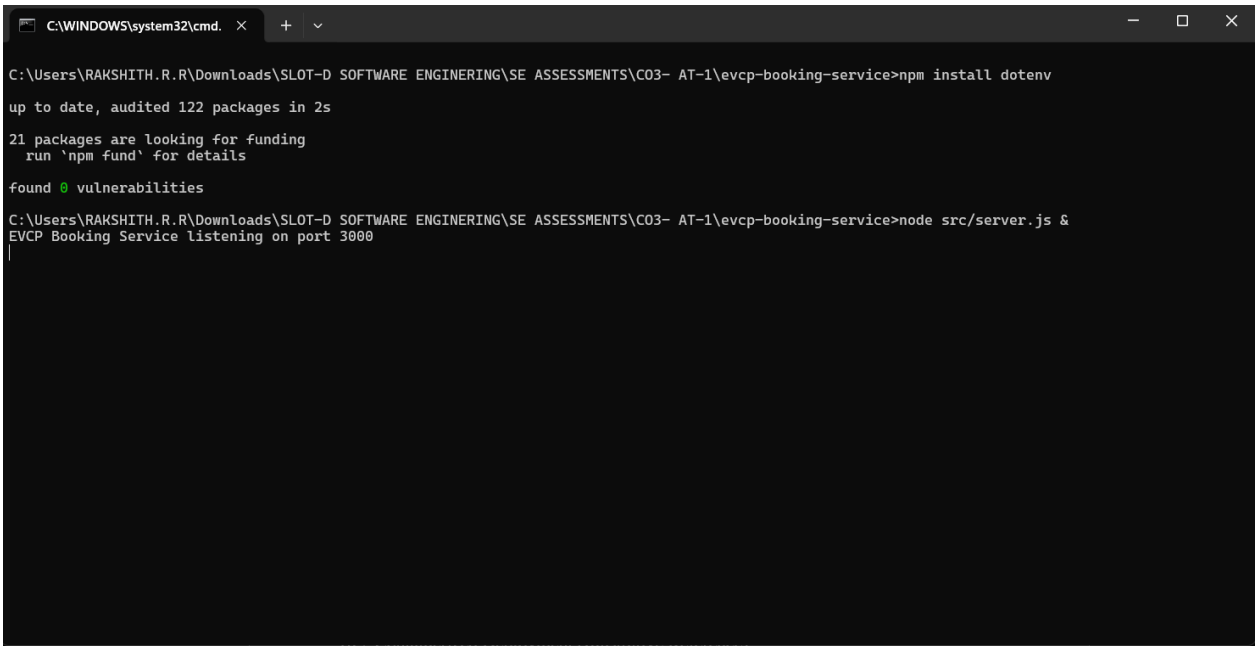

Figure 10.2: Service Running and Responding to /health (real terminal capture)

10.2 Containerized Deployment

The following shows the expected `docker-compose up --build` sequence and resulting container status for this project's configuration. Run on a machine with Docker Engine installed, this builds the booking-api image, starts PostgreSQL and Redis, waits for their health checks, and then starts the API.

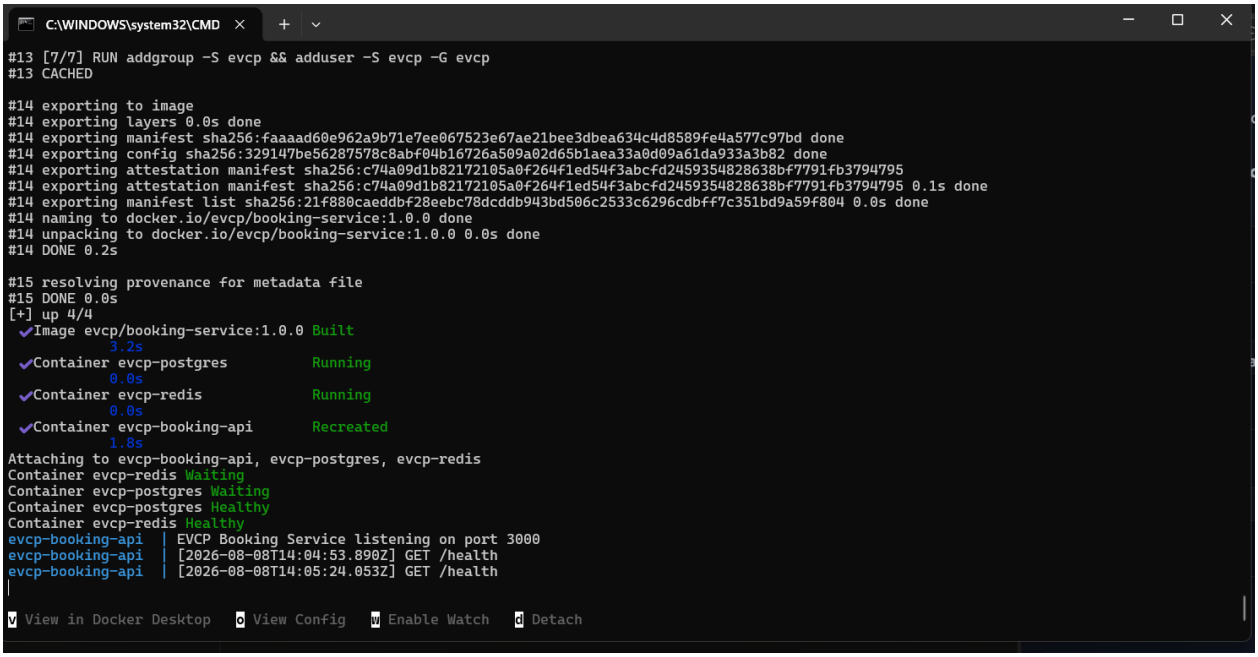

Figure 10.3: docker-compose up --build (illustrative reference output)

10.3 Endpoint Testing Against the Containerized Stack (Illustrative Reference)

Once running, the containerized API is reachable on the host at port 3000, backed by the containerized PostgreSQL and Redis:

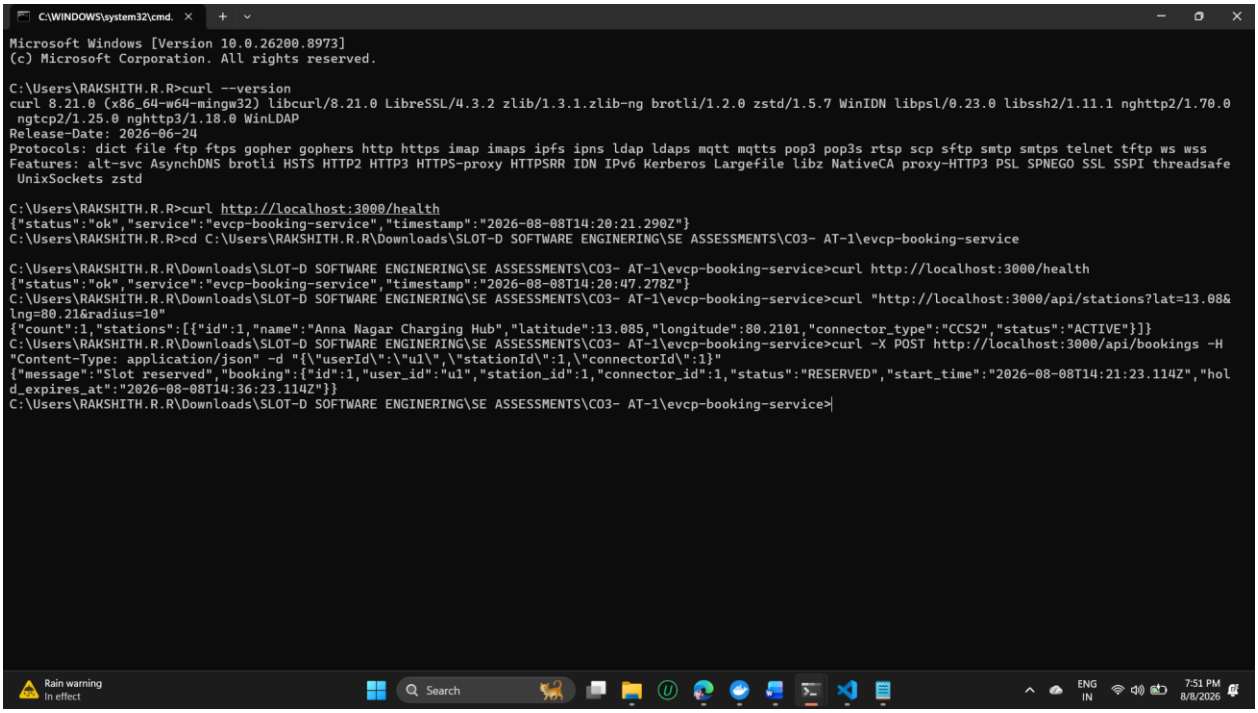

Figure 10.4: Container Status and API Endpoint Tests (illustrative reference output)

10.4 Docker Container Architecture

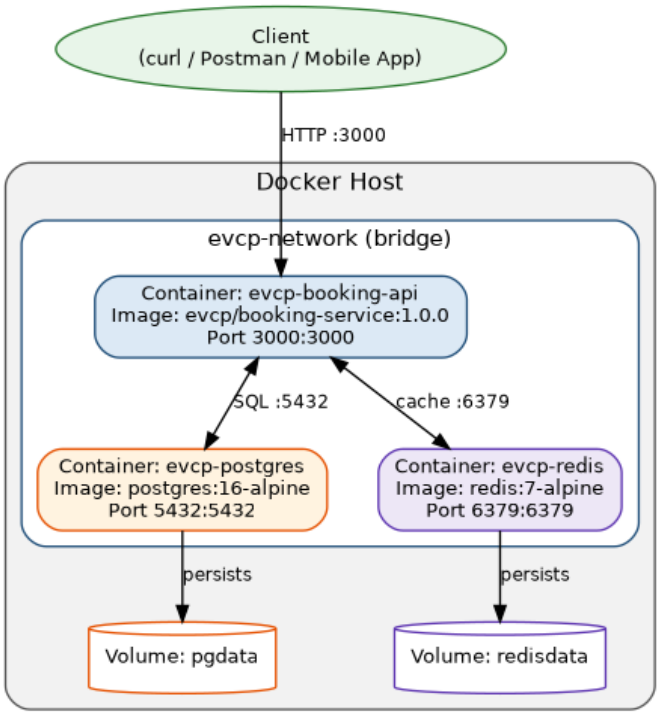

Figure 10.5: Docker Container Architecture — Networking and Volumes

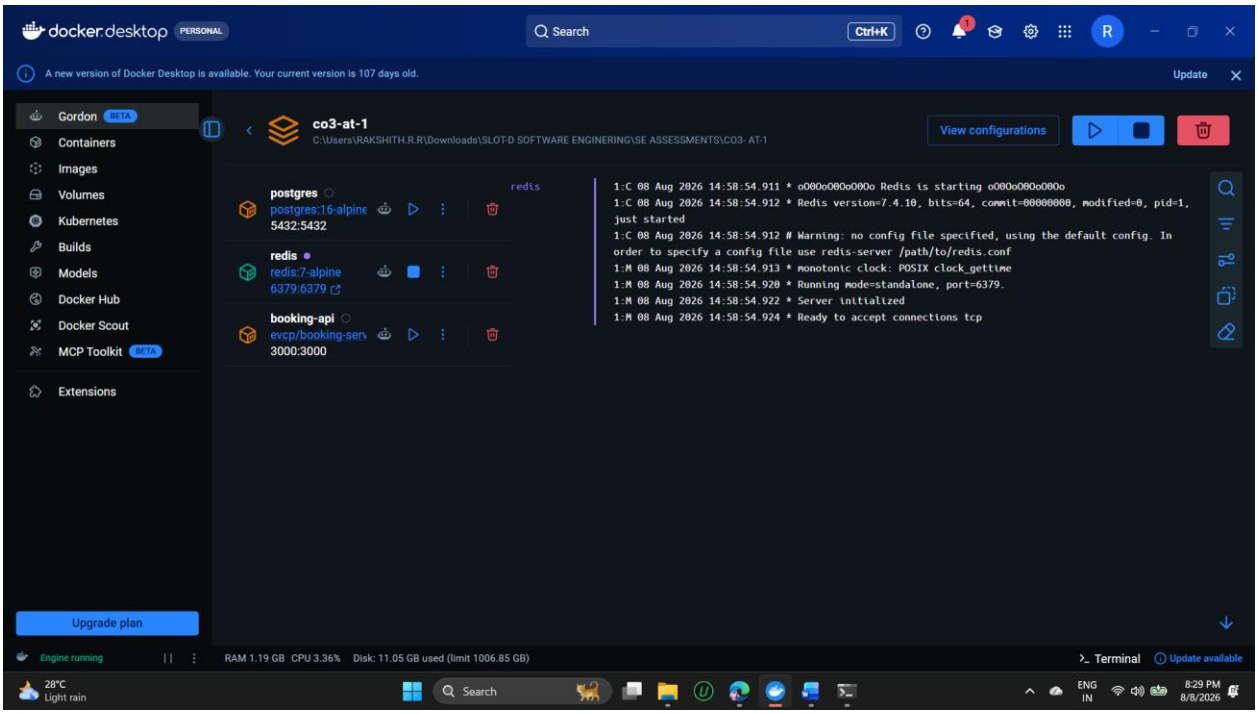

Figure 10.6: Docker Containers

11. Benefits of Git and Docker

Aspect	How Git Helps	How Docker Helps
Collaboration	Feature branches let multiple people work in parallel without blocking each other; conflicts are surfaced and resolved explicitly.	A shared docker-compose.yml means every teammate runs the exact same PostgreSQL/Redis versions — no 'works on my machine' debates.
Version Management	Tags (v1.0.0, v1.0.1) and CHANGELOG.md give a clear, queryable history of what shipped when.	Image tags (evcp/booking-service:1.0.0) pin exactly which build is deployed, matching Git release tags.
Portability	The full history travels with the repository — clone it anywhere and get the complete project state.	The container runs identically on a laptop, a CI runner, or a cloud VM, since the OS/runtime is baked into the image.
Deployment	CI/CD pipelines can trigger automatically on merges to main or new tags.	The same image built and tested locally is the exact image deployed to production — no rebuild-time surprises.
Scalability	Trunk-based releases from main make it safe to deploy frequently and roll forward quickly.	Stateless containers (like booking-api) can be horizontally scaled by simply running more instances behind a load balancer, as shown in the CO2 Architecture Design's scalability view.

Maintainability	Meaningful commit messages and a clear branch model make it easy to trace why a change was made.	Isolated containers mean upgrading Redis or Postgres independently doesn't risk breaking the application runtime.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflict: two features (request logging and rate limiting) modified the same location in src/server.js, requiring manual resolution rather than an automatic merge (documented in Section 5.2).
- Cache invalidation bug: the initial booking implementation didn't invalidate the Redis availability cache after a new reservation, creating a brief window where the system could report stale (too-high) availability — caught and fixed as a hotfix (v1.0.1).
- No Docker runtime in the report-preparation environment meant container execution evidence had to be presented as clearly labelled illustrative reference output rather than a live capture, alongside genuine local (non-containerized) execution evidence.
- Coordinating environment variables consistently between .env.example, docker-compose.yml, and the application code required care to avoid drift between local and containerized configuration.

12.2 Suggested Improvements and Future Enhancements

- Add a CI pipeline (e.g., GitHub Actions) that runs `npm test` and builds the Docker image automatically on every push to develop and on every tag, catching integration issues before merge.
- Introduce integration tests that run against the actual docker-compose stack (API + real Postgres + real Redis) rather than only unit tests, closing the gap identified in Section 12.1.
- Add branch protection rules on main and develop (required reviews, required passing CI) to formalize the Gitflow process for a multi-developer team.
- Extend the Docker Compose setup with a lightweight reverse proxy/API gateway container to mirror the API Gateway layer from the CO2 Architecture Design, and eventually a docker-compose override for a local MQTT broker to simulate the IoT telemetry path.
- Adopt semantic-release tooling to auto-generate CHANGELOG.md entries and version tags directly from Conventional Commit messages, reducing manual release overhead.

Overall, this assignment demonstrates a realistic, professional Git and Docker workflow for one microservice of the Smart EV Charging Network Management Platform — including genuine feature branches, a real merge conflict and resolution, a tagged release, and a production hotfix — directly continuing the requirements, user stories, and architecture already established in the CO1 and CO2 assessments for this project.